

----- Questions -----

Level 1 : Basics

Q1. Retrieve Customer names and emails for email marketing

select Name, email from customers;

```
52      # Level 1 : Basics
53      # Q1. Retrieve Customer names and emails for email marketing
54      select Name, email from customers;
```

	Name	email
▶	Thomas Owens	user1@example.com
	Charles Grant	user2@example.com
	Kaitlin Richards	user3@example.com
	Christina Williams	user4@example.com
	David Allen	user5@example.com
	Mark Duke	user6@example.com
	Briana Wright	user7@example.com
	John Bryan	user8@example.com
	Jason Thompson	user9@example.com
	Shawn Hill	user10@example.com
	Walter Jenkins	user11@example.com
	Mary Knight	user12@example.com
	Leslie Wilson	user13@example.com
	Deborah Arias	user14@example.com
	Austin Flores	user15@example.com
	Amy Landry	user16@example.com
	Randy Mooney	user17@example.com
	Jeffrey Bray	user18@example.com
	Amanda Bright	user19@example.com
	Megan Lee	user20@example.com
	Thomas Owens	user21@example.com

customers 186 x

Q2. View complete product catalog with all available details

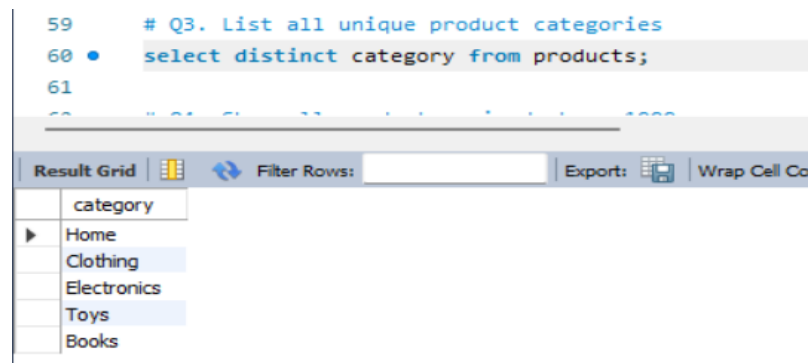
select * from products;

```
56      # Q2. View complete product catalog with all available details
57      select * from products;
```

	product_id	name	category	price	stock_quantity	added_on
▶	1	Plant No	Home	639.43	152	2024-01-30 06:30:53
	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	9	Common Study	Toys	985.19	171	2023-07-20 13:06:42
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
	13	Full West	Books	2112.33	172	2023-09-15 03:13:38
	14	Listen Development	Home	296.17	132	2024-10-12 11:49:26
	15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
	16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
	17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
	18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
	19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
	20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
	21	Red Including	Clothing	3845.21	62	2025-01-21 11:52:40

Q3. List all unique product categories

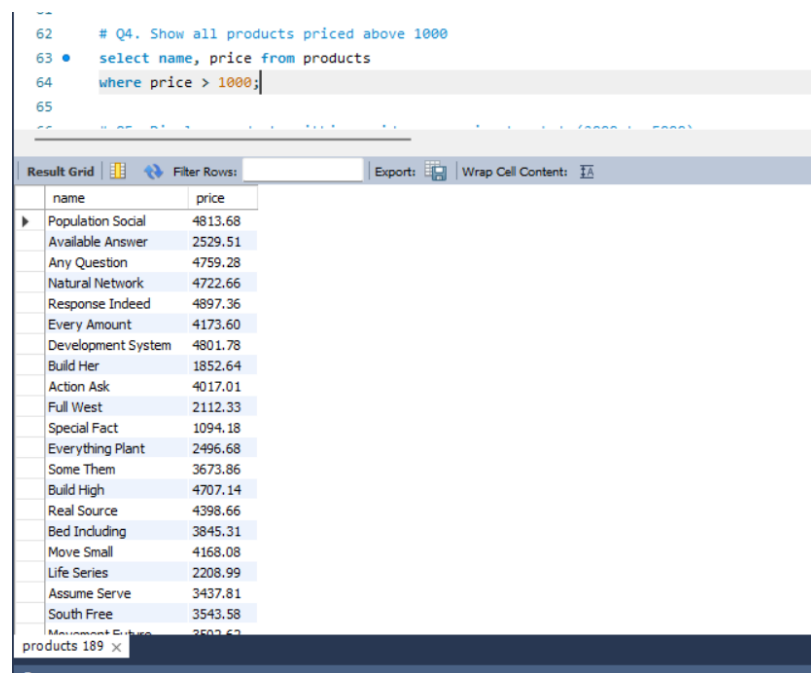
select distinct category from products;



Q4. Show all products priced above 1000

select name, price from products

where price > 1000;



Q5. Display products within a mid_range price bracket (2000 to 5000)

select name, price from products

where price between 2000 and 5000;

```

65
66 # Q5. Display products within a mid_range price bracket (2000 to 5000)
67 • select name, price from products
68 where price between 2000 and 5000;
69
70 # Q6. Fetch data for specific Customer IDs

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	name	price		
►	Population Social	4813.68		
	Available Answer	2529.51		
	Any Question	4759.28		
	Natural Network	4722.66		
	Response Indeed	4897.36		
	Every Amount	4173.60		
	Development System	4801.78		
	Action Ask	4017.01		
	Full West	2112.33		
	Everything Plant	2496.68		
	Some Them	3673.86		
	Build High	4707.14		
	Real Source	4398.66		
	Bed Including	3845.31		
	Move Small	4168.08		
	Life Series	2208.99		
	Assume Serve	3437.81		
	South Free	3543.58		
	Movement Future	3502.62		
	East Foot	2414.93		
	Free Developer	4160.45		

products 190 x

Q6. Fetch data for specific Customer IDs

```
select * from customers
```

```
where customer_id in (1,10,14,24);
```

here i extracts data of random customers.

```

70 # Q6. Fetch data for specific Customer IDs
71 • select * from customers
72 where customer_id in (1,10,14,24);
73 # here i extracts data of random customers.
74
75 # Q7. Identify customers whose name starts with the letter "A"
76 • select * from customers

```

Result Grid		Filter Rows:	Edit:	Export/Import:	Wrap Cell
	customer_id	name	email	phone	created_at
►	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
	10	Shawn Hill	user10@example.com	(268) 113-3152x7	2023-12-14 20:46:43
	14	Deborah Arias	user14@example.com	811-821-2144x97	2024-04-24 00:27:28
	24	Brandy Wright	user24@example.com	853.520.8915x61	2024-07-09 17:09:19
*	NULL	NULL	NULL	NULL	NULL

Q7. Identify customers whose name starts with the letter "A"

```
select * from customers
```

```
where name like "A%";
```

```

74
75 # Q7. Identify customers whose name starts with the letter "A"
76 • select * from customers
77 where name like "A%";
78
79 # Q8. List electronic Products priced under 3000
80 • select * from products
81 where category = "Electronics" and price < 3000;
82

```

	customer_id	name	email	phone	created_at
▶	15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
	16	Amy Landry	user16@example.com	+1-278-019-3748	2024-02-28 17:51:50
	19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15
	27	Adrienne Green	user27@example.com	530.644.8455x93	2023-08-22 01:55:29
•	NULL	NULL	NULL	NULL	NULL

Q8. List electronic Products priced under 3000

select * from products

where category = "Electronics" and price < 3000;

```

79 # Q8. List electronic Products priced under 3000
80 • select * from products
81 where category = "Electronics" and price < 3000;
82

```

	product_id	name	category	price	stock_quantity	added_on
▶	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
	15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
	31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
	34	Despite Win	Electronics	1340.34	64	2024-11-27 06:55:45
	44	Actually Term	Electronics	396.11	85	2023-11-02 13:09:20
	47	Southern Thing	Electronics	512.46	40	2024-02-28 17:57:38
•	NULL	NULL	NULL	NULL	NULL	NULL

Q9. Display Products name and price in descending order of price

select name, price from products

order by price desc;

```

83      # Q9. Display Products name and price in descending order of price
84      • select name, price from products
85      order by price desc;
86

```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	name	price			
▶	Response Indeed	4897.36			
	Population Social	4813.68			
	Development System	4801.78			
	Any Question	4759.28			
	Fire Often	4734.89			
	Natural Network	4722.66			
	Build High	4707.14			
	Serious Recognize	4523.10			
	Study Total	4413.68			
	Real Source	4398.66			
	Involve Officer	4244.09			
	Group But	4180.23			
	Every Amount	4173.60			

Q10 Display products names and prices, sorted by price and then by name

select name, price from products

order by price desc, name ;

```

86
87      # Q10 Display products names and prices, sorted by price and then by name
88      • select name, price from products
89      order by price desc, name ;
90

```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	name	price			
▶	Response Indeed	4897.36			
	Population Social	4813.68			
	Development System	4801.78			
	Any Question	4759.28			
	Fire Often	4734.89			
	Natural Network	4722.66			
	Build High	4707.14			
	Serious Recognize	4523.10			
	Study Total	4413.68			
	Real Source	4398.66			
	Involve Officer	4244.09			
	Group But	4180.23			
	Every Amount	4173.60			

Level 2: Filtering and Formatting

Q1. Retrieve orders where customer information is missing

select * from orders

where customer_id is null;

```

92      # LEVEL 2: FILTERING AND FORMATTING
93
94      # Q1. Retrieve orders where customer information is missing (possibly due to data migration or deletion)
95      • select * from orders
96      where customer_id is null;
97
98      # Q2. Display customer names and emails using column aliases for frontend readability

```

order_id	customer_id	order_date	status	total_amount
NULL	NULL	NULL	NULL	NULL

Q2. Display customer names and emails using column aliases for frontend readability

select name as Customer_Name, email as Email from customers;

```

97
98      # Q2. Display customer names and emails using column aliases for frontend readability
99      • select name as Customer_Name, email as Email from customers;
100

```

Customer_Name	Email
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com
Jason Thompson	user9@example.com
Shawn Hill	user10@example.com
Walter Jenkins	user11@example.com
Mary Knight	user12@example.com
Leslie Wilson	user13@example.com

Q3. Calculate total value per item ordered by multiplying quantity and item price

select quantity, item_price, (quantity * item_price) as Total_value

from order_items;

```

100
101      # Q3. Calculate total value per item ordered by multiplying quantity and item price
102      • select quantity, item_price, (quantity * item_price) as Total_value
103      from order_items;
104

```

quantity	item_price	Total_value
2	4707.14	9414.28
3	177.40	532.20
3	985.19	2955.57
1	2208.99	2208.99
2	4734.89	9469.78
1	4707.14	4707.14
2	4897.36	9794.72
3	4897.36	14692.08
1	1429.45	1429.45
1	723.97	723.97
3	4734.89	14204.67
1	3043.67	3043.67
3	4801.78	14405.34

Q4. Combine Customer name and phone number in a single column

```
select concat(name," : ",phone) as Customer_Details

from customers;
```

105 # Q4. Combine Customer name and phone number in a single column
 106 • select concat(name," : ",phone) as Customer_Details
 107 from customers;
 108

Customer_Details
Thomas Owens : 142-479-1945
Charles Grant : 9153947511
Kaitlin Richards : 2073473421
Christina Williams : 586-605-5061x06
David Allen : (751)456-8289x1
Mark Duke : (144)957-2811
Briana Wright : 223-833-9635
John Bryan : 045.568.0798x27
Jason Thompson : 1862659420
Shawn Hill : (268)113-3152x7
Walter Jenkins : 536-329-0817x71
Mary Knight : 361-636-3802
Leslie Wilson : +1-256-261-1984

Result 200

Q5. Extract only the date part from order timestamps for date-wise reporting

```
select date(order_date) as Date from orders;
```

108
 109 # Q5. Extract only the date part from order timestamps for date-wise reporting
 110 • select date(order_date) as Date from orders;
 111

Date
2025-03-02
2024-10-09
2025-05-08
2024-09-19
2025-04-08
2024-10-25
2024-07-29
2024-07-30
2025-06-10
2025-02-16
2025-03-11
2025-01-15
2025-05-12

Q6. List products that do not have any stock left.

```
select * from products

where stock_quantity = 0;
```

111
 112 # Q6. List products that do not have any stock left.
 113 • select * from products
 114 where stock_quantity = 0;
 115

product_id	name	category	price	stock_quantity	added_on
NULL	NULL	NULL	NULL	NULL	NULL

Level 3: Aggregation

Q1. Count the total number of orders placed

`select count(*) as Total_Orders from orders;`

```
116
117         # Level 3: Aggregation
118
119     # Q1. Count the total number of orders placed
120 •   select count(*) as Total_Orders from orders;
121
```

Total_Orders
400

Q2. Calculate the total revenue collected from all orders

`select sum(total_amount) as Total_Revenue from orders;`

```
121
122     # Q2. Calculate the total revenue collected from all orders
123 •   select sum(total_amount) as Total_Revenue from orders;
124
```

Total_Revenue
6960973.66

Q3. calculate the avg of value.

`select avg(total_amount) as Average_Order_Value from orders;`

```
124
125     # Q3. calculate the avg of value.
126 •   select avg(total_amount) as Average_Order_Value from orders;
127
```

Average_Order_Value
17402.434150

Q4. Count the number of customers who have placed at least one orders

`select count(distinct customer_id) as Customers from orders;`


```

127
128 # Q4. Count the number of customers who have placed at least one orders
129 • select count(distinct customer_id) as Customers from orders;
130

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
Customers			
30			

Q5. Find the nummber of orders placed by each customer

```

select customer_id, count(customer_id) as total_orders from orders
group by customer_id;

```

```

130
131 # Q5. Find the nummber of orders placed by each customer
132 • select customer_id, count(customer_id) as total_orders from orders
133 group by customer_id;
134
135 # Q6. Find total sales amount made by each customer

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
customer_id	total_orders		
1	12		
2	17		
3	17		
4	9		
5	14		
6	16		
7	13		
8	10		
9	10		
10	13		
11	9		
12	11		
--	--		

Q6. Find total sales amount made by each customer

```

select customer_id, sum(total_amount) as Total_Sales from orders
group by customer_id;

```

```

134
135 # Q6. Find total sales amount made by each customer
136 • select customer_id, sum(total_amount) as Total_Sales from orders
137 group by customer_id;
138

```

customer_id	Total_Sales
1	183747.44
2	284420.07
3	253783.31
4	137562.22
5	262504.19
6	212173.58
7	167960.11
8	164701.78
9	106226.03
10	252722.75
11	173409.01
12	169653.79
13	152727.70

Result 209 x

Q7. List the number of products sold per category.

select category, count(*) as Items from products join order_items

on products.product_id = order_items.product_id

group by category;

```

138
139 # Q7. List the number of products sold per categoryT
140 • select category, count(*) as Items from products join order_item
141 on products.product_id = order_items.product_id
142 group by category;
143

```

category	Items
Home	216
Clothing	281
Electronics	333
Toys	201
Books	170

Q8. Find the average item price per category

select category, avg(price) as Average_Price from products

group by category;

```

144
145 # Q8. Find the average item price per category
146 • select category, avg(price) as Average_Price from product;
147 group by category;
148

```

category	Average_Price
Home	2146.367500
Clothing	3434.581538
Electronics	2653.388571
Toys	2516.526250
Books	3167.084286

Q9. Show number of orders placed per day

```

select date(order_date) as Date,count(*) as Orders
from orders
group by date(order_date)
order by date(order_date);

```

```

148
149 # Q9. Show number of orders placed per day
150 • select date(order_date) as Date,count(*) as Orders
151 from orders
152 group by date(order_date)
153 order by date(order_date);
154
155 # Q10. List total payments received per payment method

```

Date	Orders
2024-06-15	1
2024-06-18	1
2024-06-19	3
2024-06-20	2
2024-06-21	2
2024-06-23	2
2024-06-24	1

Q10. List total payments received per payment method

```

select Method,sum(amount_paid) as Total_Price
from payments
group by method;

```

```

153     order by date(order_date),
154
155     # Q10. List total payments received per payment method
156     select Method,sum(amount_paid) as Total_Price
157     from payments
158     group by method;
159

```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 

	Method	Total_Price
▶	Credit Card	1754603.10
	Net Banking	1658383.90
	UPI	1617408.78
	Debit Card	1930577.88

Level 4: Multi-Table Queries (JOIN)

Q1. Retrieve order details along with the customer name (INNER JOIN)

select * from customers

inner join orders

on customers.customer_id = orders.customer_id;

```

160
161     # Level 4: Multi-Table Queries (JOIN)
162
163     # Q1. Retrieve order details along with the customer name (INNER JOIN)
164     select * from customers
165     inner join orders
166     on customers.customer_id = orders.customer_id;
167

```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 

	customer_id	name	email	phone	created_at	order_id	customer_id	order_date	status	total_amount
▶	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	14	1	2024-09-24 21:21:38	Shipped	15803.34
	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	17	1	2024-08-19 21:17:57	Pending	11173.77
	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	61	1	2024-12-26 23:21:58	Shipped	13053.00
	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	76	1	2025-01-14 22:59:54	Pending	23506.81
	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	92	1	2024-09-26 11:33:58	Shipped	834.75
	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	109	1	2025-03-17 04:56:18	Shipped	12190.14
	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	127	1	2025-06-10 18:01:32	Pending	20160.64
	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	135	1	2025-03-11 22:58:23	Pending	8891.79
	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	144	1	2024-09-19 09:18:22	Shipped	12093.54
	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	221	1	2024-09-25 05:49:39	Pending	12437.61
	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	278	1	2025-02-01 20:05:38	Delivered	32015.16
	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	379	1	2025-03-22 15:57:58	Cancelled	21586.89
	2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24	56	2	2024-06-23 15:59:57	Pending	6828.61

Result 214 x

Q2. Get list of products that have been sold (INNER JOIN with order_items)

select * from products

inner join order_items

on products.product_id = order_items.product_id;

```

167
168 # Q2. Get list of products that have been sold (INNER JOIN with order_items)
169 • select * from products
170 inner join order_items
171 on products.product_id = order_items.product_id;
172

```

Result Grid											
Filter Rows:											
Export: Wrap Cell Content:											
	product_id	name	category	price	stock_quantity	added_on	order_item_id	order_id	product_id	quantity	item_price
▶	1	Plant No	Home	639.43	152	2024-01-30 06:30:53	20	10	1	1	639.43
	1	Plant No	Home	639.43	152	2024-01-30 06:30:53	23	11	1	3	639.43
	1	Plant No	Home	639.43	152	2024-01-30 06:30:53	72	28	1	1	639.43
	1	Plant No	Home	639.43	152	2024-01-30 06:30:53	103	37	1	1	639.43
	1	Plant No	Home	639.43	152	2024-01-30 06:30:53	140	49	1	2	639.43
	1	Plant No	Home	639.43	152	2024-01-30 06:30:53	210	71	1	3	639.43
	1	Plant No	Home	639.43	152	2024-01-30 06:30:53	223	75	1	2	639.43
	1	Plant No	Home	639.43	152	2024-01-30 06:30:53	248	81	1	2	639.43
	1	Plant No	Home	639.43	152	2024-01-30 06:30:53	372	127	1	2	639.43
	1	Plant No	Home	639.43	152	2024-01-30 06:30:53	398	134	1	2	639.43
	1	Plant No	Home	639.43	152	2024-01-30 06:30:53	493	171	1	2	639.43
	1	Plant No	Home	639.43	152	2024-01-30 06:30:53	682	236	1	3	639.43
	1	Plant No	Home	639.43	152	2024-01-30 06:30:53	695	239	1	3	639.43

Q3. List all orders with their payment method (INNER JOIN)

select payment_id as Order_ID, method from orders

inner join payments

on orders.order_id = payments.order_id;

```

172
173 # Q3. List all orders with their payment method (INNER JOIN )
174 • select payment_id as Order_ID, method from orders
175 inner join payments
176 on orders.order_id = payments.order_id;
177
178 # Q4. Get list of customers and their orders (LEFT JOIN)

```

Result Grid											
Filter Rows:											
Export: Wrap Cell Content:											
	Order_ID	method									
▶	14	UPI									
	17	UPI									
	61	Net Banking									
	76	Credit Card									
	92	Net Banking									
	109	UPI									
	127	Debit Card									
	135	Debit Card									
	144	UPI									
	221	UPI									
	278	UPI									
	379	Credit Card									

Q4. Get list of customers and their orders (LEFT JOIN)

select * from customers

left join orders

on customers.customer_id = orders.customer_id;

```

177
178 # Q4. Get list of customers and their orders (LEFT JOIN)
179 • select * from customers
180 left join orders
181 on customers.customer_id = orders.customer_id;
182
183 # Q5. List all products along with order item quantity (LEFT JOIN)

```

customer_id	name	email	phone	created_at	order_id	customer_id	order_date	status	total_amount
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	14	1	2024-09-24 21:21:38	Shipped	15803.34
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	17	1	2024-08-19 21:17:57	Pending	11173.77
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	61	1	2024-12-26 23:21:58	Shipped	13053.00
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	76	1	2025-01-14 22:59:54	Pending	23506.81
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	92	1	2024-09-26 11:33:58	Shipped	834.75
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	109	1	2025-03-17 04:56:18	Shipped	12190.14
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	127	1	2025-06-10 18:01:32	Pending	20160.64
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	135	1	2025-03-11 22:58:23	Pending	8891.79
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	144	1	2024-09-19 09:18:22	Shipped	12093.54
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	221	1	2024-09-25 05:49:39	Pending	12437.61
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	278	1	2025-02-01 20:05:38	Delivered	32015.16
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	379	1	2025-03-22 15:57:58	Cancelled	21586.89
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24	56	2	2024-06-23 15:59:57	Pending	6828.61

Result 217 x

Q5. List all products along with order item quantity (LEFT JOIN)

select name, count(name) as Order_Item_Quantity from products

left join order_items

on products.product_id = order_items.product_id

group by name;

```

182
183 # Q5. List all products along with order item quantity (LEFT JOIN)
184 • select name, count(name) as Order_Item_Quantity from products
185 left join order_items
186 on products.product_id = order_items.product_id
187 group by name;
188

```

name	Order_Item_Quantity
Plant No	22
Population Social	16
Available Answer	25
Any Question	13
Natural Network	21
If Whatever	27
Response Indeed	21
Every Amount	27
Common Study	26
Development System	23
Build Her	24
Action Ask	25
Full West	20

Q6. List all payments including those with no matching orders (RIGHT JOIN)

select method, order_item_id from payments

right join order_items

on payments.order_id = order_items.order_id;

```

100
189 # Q6. List all payments including those with no matching orders (RIGHT JOIN)
190 • select method, order_item_id from payments
191 right join order_items
192 on payments.order_id = order_items.order_id;
193

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
method	order_item_id			
Credit Card	1			
Net Banking	2			
Credit Card	3			
Credit Card	4			
UPI	5			
UPI	6			
UPI	7			
UPI	8			
UPI	9			
UPI	10			
UPI	11			
UPI	12			
Net Banking	13			

Result 219

Q7. Combine data from three tables: customer, order, and payment

select * from customers

join orders

on customers.customer_id = orders.customer_id

join payments

on orders.order_id = payments.order_id;

```

193
194 # Q7. Combine data from three tables: customer, order, and payment
195 • select * from customers
196 join orders
197 on customers.customer_id = orders.customer_id
198 join payments
199 on orders.order_id = payments.order_id;
200
201

```

Result Grid		Filter Rows:		Export:		Wrap Cell Content:								
customer_id	name	email	phone	created_at	order_id	customer_id	order_date	status	total_amount	payment_id	order_id	payment_date	amount_paid	method
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	14	1	2024-09-24 21:21:38	Shipped	15803.34	14	14	2024-09-24 22:11:38	15803.34	UPI
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	17	1	2024-08-19 21:17:57	Pending	11173.77	17	17	2024-08-19 21:49:57	11173.77	UPI
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	61	1	2024-12-26 23:21:58	Shipped	13053.00	61	61	2024-12-26 23:31:58	13053.00	Net Banking
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	76	1	2025-01-14 22:59:54	Pending	23506.81	76	76	2025-01-14 23:33:54	23506.81	Credit Card
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	92	1	2024-09-26 11:33:58	Shipped	834.75	92	92	2024-09-26 12:03:58	834.75	Net Banking
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	109	1	2025-03-17 04:56:18	Shipped	12190.14	109	109	2025-03-17 05:16:18	12190.14	UPI
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	127	1	2025-06-10 18:01:32	Pending	20160.64	127	127	2025-06-10 18:57:32	20160.64	Debit Card
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	135	1	2025-03-11 22:58:23	Pending	8891.79	135	135	2025-03-11 23:11:23	8891.79	Debit Card
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	144	1	2024-09-19 09:18:22	Shipped	12093.54	144	144	2024-09-19 10:10:22	12093.54	UPI
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	221	1	2024-09-25 05:49:39	Pending	12437.61	221	221	2024-09-25 06:25:39	12437.61	UPI
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	278	1	2025-02-01 20:05:38	Delivered	32015.16	278	278	2025-02-01 21:05:38	32015.16	UPI
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	379	1	2025-03-22 15:57:58	Cancelled	21586.89	379	379	2025-03-22 16:48:58	21586.89	Credit Card

Result 220

Read

Level 5: Subqueries (Inner Queries)

Q1. List all products priced above the average product price

select Name, Price from products

where price > (select avg(price) from products);

```
201
202           # Level 5: Subqueries (Inner Queries)
203
204   # Q1. List all products priced above the average product price
205 •   select Name, Price from products
206      where price > (select avg(price) from products);
207
```

Name	Price
Population Social	4813.68
Any Question	4759.28
Natural Network	4722.66
Response Indeed	4897.36
Every Amount	4173.60
Development System	4801.78
Action Ask	4017.01
Some Them	3673.86
Build High	4707.14
Real Success	4300.66

Q2. Find customers who have placed at least one order

select name, count(name) as orders from customers

inner join orders

on customers.customer_id = orders.customer_id

group by name;

```
207
208   # Q2. Find customers who have placed at least one order
209 •   select name, count(name) as orders from customers
210      inner join orders
211      on customers.customer_id = orders.customer_id
212      group by name;
213
214   # Q3. Show orders whose total amount is above the average for that customer
```

name	orders
Thomas Owens	12
Charles Grant	17
Kaitlin Richards	17
Christina Williams	9
David Allen	14
Mark Duke	16
Briana Wright	13
John Bryan	10
Jason Thompson	10
Shawn Hill	13

Q3. Show orders whose total amount is above the average for that customer

select Name, order_id, total_amount from orders

inner join customers

on customers.customer_id = orders.customer_id

where total_amount > (select avg(total_amount) from orders);

```
213
214 # Q3. Show orders whose total amount is above the average for that customer
215 • select Name, order_id, total_amount from orders
216 inner join customers
217 on customers.customer_id = orders.customer_id
218 where total_amount > (select avg(total_amount) from orders);
219
```

Result Grid			
Filter Rows:			
Export: Wrap Cell Content:			
Name	order_id	total_amount	
Kara Zavala	6	31050.17	
Amanda Bright	8	32714.06	
Mark Duke	9	24219.20	
Joseph Stuart	10	24342.52	
David Allen	16	22856.73	
Leslie Wilson	18	32001.24	
Peter Phillips	21	25364.11	
Megan Lee	22	21281.44	
Cindy Hart	23	18045.05	
Mark Duke	24	22882.19	
Randy Mooney	25	23749.66	
Brandy Wright	26	35167.92	

Q4. Display customers who haven't placed any orders

SELECT *

FROM customers

LEFT JOIN orders

ON customers.customer_id = orders.customer_id

where order_id is null;

```
219
220 # Q4. Display customers who haven't placed any orders
221 • SELECT *
222 FROM customers
223 LEFT JOIN orders
224 ON customers.customer_id = orders.customer_id
225 where order_id is null;
226
```

Result Grid									
Filter Rows:									
Export: Wrap Cell Content:									
customer_id	name	email	phone	created_at	order_id	customer_id	order_date	status	total_amount

Q5. Show products that were never ordered

select * from products

LEFT JOIN order_items

ON products.product_id = order_items.product_id

WHERE order_items.product_id IS NULL;

```
227
228 # Q5. Show products that were never ordered
229 • select * from products
230 LEFT JOIN order_items
231 ON products.product_id = order_items.product_id
232 WHERE order_items.product_id IS NULL;
233
```

product_id	name	category	price	stock_quantity	added_on	order_item_id	order_id	product_id	quantity	item_price
------------	------	----------	-------	----------------	----------	---------------	----------	------------	----------	------------

Q6. Show highest value order per customer

select name, max(total_amount) as Highest_Amount from customers

join orders

on customers.customer_id = orders.customer_id

group by name;

```
234 WHERE order_items.product_id IS NULL;
233
234 # Q6. Show highest value order per customer
235 • select name, max(total_amount) as Highest_Amount from customers
236 join orders
237 on customers.customer_id = orders.customer_id
238 group by name;
239
240 # Q7. Highest Order Per Customer (Including Names)
```

name	Highest_Amount
Thomas Owens	32015.16
Charles Grant	42056.04
Kaitlin Richards	41679.11
Christina Williams	25747.34
David Allen	39921.78
Mark Duke	39003.19
Briana Wright	28589.04
John Bryan	36147.09
Jason Thompson	21414.44
Shawn Hill	35723.17
Walter Jenkins	37129.82
Mary Knight	38656.26
Leslie Wilson	32001.24

Q7. Highest Order Per Customer (Including Names)

select name, max(total_amount) as Highest_Amount from customers

join orders

on customers.customer_id = orders.customer_id

group by name;

```

239
240 # Q7. Highest Order Per Customer (Including Names)
241 • select name, max(total_amount) as Highest_Amount from customers
242 join orders
243 on customers.customer_id = orders.customer_id
244 group by name;
245
246

```

Result Grid
Filter Rows:
Export:
Wrap Cell Content:

	name	Highest_Amount
▶	Thomas Owens	32015.16
	Charles Grant	42056.04
	Kaitlin Richards	41679.11
	Christina Williams	25747.34
	David Allen	39921.78
	Mark Duke	39003.19
	Briana Wright	28589.04
	John Bryan	36147.09
	Jason Thompson	21414.44
	Shawn Hill	35723.17
	Walter Jenkins	37129.82
	Mary Knight	38656.26
	Leslie Wilson	32001.24

Result 227 ✕

Output

Level 6: Set Operations

Q1. List all customers who have either placed an order or written a product review

```

select *
from customers
where customer_id in ( select customer_id from orders)
union
select * from customers
where customer_id in ( select customer_id from product_reviews);

```

```

246
247         # Level 6: Set Operations
248
249     # Q1. List all customers who have either placed an order or written a product review
250     select *
251     from customers
252     where customer_id in ( select customer_id from orders)
253     union
254     select * from customers
255     where customer_id in ( select customer_id from product_reviews);
256

```

Result Grid					
Filter Rows:					
Export:					
Wrap Cell Content:					
	customer_id	name	email	phone	created_at
▶	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
	2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
	3	Kaitlin Richards	user3@example.com	2073473421	2024-06-23 09:55:22
	4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
	5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
	6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59
	7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43
	8	John Bryan	user8@example.com	045.568.0798x27	2025-02-15 17:57:04
	9	Jason Thompson	user9@example.com	1862659420	2024-08-31 08:18:51
	10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43
	11	Walter Jenkins	user11@example.com	536-329-0817x71	2023-10-26 03:12:30
	12	Mary Knight	user12@example.com	361-636-3802	2023-08-16 20:05:50
	13	Leslie Wilson	user13@example.com	+1-256-261-1984	2024-06-06 20:12:35

Q2. List all customers who have placed an order as well as reviewed

```
select * from customers
```

```
where exists ( select 1 from orders
```

```
where customers.customer_id = orders.customer_id
```

```
)
```

```
and
```

```
exists ( select 1 from product_reviews
```

```
where product_reviews.customer_id = customers.customer_id
```

```
);
```

```

256
257 # Q2. List all customers who have placed an order as well as reviewed
258 • select * from customers
259   where exists ( select 1 from orders
260                 where customers.customer_id = orders.customer_id
261               )
262   and
263   exists ( select 1 from product_reviews
264           where product_reviews.customer_id = customers.customer_id
265         );
266

```

Result Grid					
Filter Rows:					
Edit:					
Export/Import:					
Wrap Cell Content:					
	customer_id	name	email	phone	created_at
▶	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
	2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
	4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
	6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59
	7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43
	9	Jason Thompson	user9@example.com	1862659420	2024-08-31 08:18:51
	10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43
	11	Walter Jenkins	user11@example.com	536-329-0817x71	2023-10-26 03:12:30
	13	Leslie Wilson	user13@example.com	+1-256-261-1984	2024-06-06 20:12:35
	14	Deborah Arias	user14@example.com	811-821-2144x97	2024-04-24 00:27:28
	15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
	17	Randy Mooney	user17@example.com	(158)927-7313x3	2023-09-21 13:23:02
	19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15

customers 229 x

Output